Lógica do jogo — Oficina Arena / Arena Bot

Disciplina: Computação Gráfica (integração com PP01 de IA)
Objetivo: mapa da lógica para melhorar o gameplay sem reescrever Minimax/A*.

---

1. Duas camadas

Camada	Onde	Papel
Regra de jogo + IA	`world/core` + `world/ia` (cópia do Arena Bot)	Decide alvo, rota, coleta, pontuação
Cena 3D	`scene/`, `app/`, OpenGL	Edita a grade e desenha o estado

O OpenGL **não** decide rota. Ele lê posição/estado e anima.

Fluxo resumido

1. Edição da cena 15×9
2. Tecla **P** / **R** → `WorldBridge` sincroniza cena → `Partida`
3. A cada ~0,35 s → `Partida.executar_turno()`
4. Decidir alvo (Minimax ou base via BC) → A* → andar 1 célula
5. Na base: descarregar; senão: tentar coletar se `Acoletar`
6. Espelhar minérios removidos na cena 3D

---

2. Loop de um turno

Arquivo: `world/core/partida.py` → `Partida.executar_turno`

1. Alterna **Alfa** / **Beta** (`turno % 2`).
2. Se bateria ≤ 0 → só passa a vez.
3. **Alvo:** reusa caminho válido; senão Minimax; se BC tem `Pdescarga` → força base.
4. **Rota:** A* até o alvo (até ~n_minérios+3 tentativas).
5. Avança **1 célula**; custo −2 (normal) ou −4 (armadilha).
6. Na base → `descarregar()` (pontos + bateria cheia).
7. Fora da base → `tentar_coletar()` só se a BC derivar `Acoletar`.
8. Se chegou no alvo e não coletou → limpa alvo e reavalia.
9. **Fim:** ambos sem bateria, ou sem minérios/entregas possíveis. Vence quem tem mais pontos.

---

3. Mundo

Peça	Regra
Grade	15×9, movimento ortogonal (N/S/L/O)
Parede	Intransitável
Armadilha	Transitável, custo dobrado

Peça	Regra
Minério	diamante 100 / rubi 80 / ouro 50 / prata 30 / bronze 10
Base Alfa	(0, 0)
Base Beta	(14, 8)
Defaults	25 paredes, 6 armadilhas, 12 minérios

- Sem colisão robô–robô.
- Decoração `LIVRE` (cadeira, planta, luminária) ocupa a cena mas **não** entra na arena → na simulação o robô "atravessa" a malha.

---

4. Estado do robô

Arquivo: `world/core/agente.py`

Campo	Papel
`posicao`, `base`	Célula atual e base própria
`bateria`	Começa em 100; −2 / −4 por passo
`carga`	0…3
`pontuacao`	+valor ao coletar; `+carga×10` ao descarregar
`alvo`, `caminho`	Meta e fila A*
`bc`	Base de conhecimento proposicional

---

5. Três camadas de decisão

5.1 Base de conhecimento (coleta / retorno)

Arquivo: `world/ia/conhecimento.py`

Observados: `Balta` (bateria > 10), `Ccheia` (carga ≥ 3), `Sminerio`, `Bcritica`, `Aadj` (observado sem regra).

Regras:

- `Balta ∧ ¬Ccheia ⇒ Pcoleta`
- `Ccheia ⇒ Pdescarga`
- `Bcritica ⇒ Pdescarga`
- `Sminerio ∧ Pcoleta ⇒ Acoletar`

5.2 Minimax α-β (qual minério / base)

Arquivo: `world/ia/estrategia.py` — profundidade **4**.

- Ações = escolher minério ou base (não cada passo).
- Pré-filtro: só minérios com bateria para ida+volta+margem; senão volta à base.
- Avaliação favorece valor do minério e vantagem de distância vs oponente.
- Alfa e Beta usam a **mesma** política.

5.3 A* (caminho célula a célula)

Arquivo: `world/ia/navegacao.py`

- Heurística: Manhattan.
- Custo de aresta: 1 (normal) / 2 (armadilha).

---

6. O que a oficina 3D muda

Ação	Efeito
Catálogo	bancada/estante → `PAREDE`; minério; armadilha; base; décor → `LIVRE`
P / Espaço	sync cena → partida (pause/resume)
R	reinicia a partir da cena
Título da janela	placar, bateria, carga, turno; no fim: vencedor e volta à edição

Gap: minérios colocados pelo usuário viram ouro genérico (valor 50) no sync.

Contrato do repositório (`AGENTS.md`): **não reescrever Minimax/A***; melhorar via ponte, HUD, placement e constantes.

---

7. Alavancas para melhorar

1. **Feedback** — placar/bateria/carga no título (já feito); overlay OpenGL se quiser.
2. **Consistência cena↔IA** — décor como parede ou não-ocupante; preservar tipo/valor do minério no sync.
3. **Ritmo visual** — `turn_interval` alinhado ao passo do robô.
4. **Balanceamento** — `world/config.py`: custos, carga, pesos Minimax, densidade do mapa.
5. **Políticas assimétricas** — só se quiser diferenciar Alfa/Beta (cuidado com o escopo de CG).

---

8. Arquivos-chave

Objetivo	Arquivo
Ponte cena↔partida	`world/bridge.py`
Turnos / fim	`world/core/partida.py`
Robô	`world/core/agente.py`
Constantes	`world/config.py`
A* / Minimax / BC	`world/ia/*.py`
Input / título	`app/application.py`
Catálogo → célula	`scene/catalog.py`
Placement	`scene/workshop.py`

Documento gerado a partir do mapa de lógica da Oficina Arena.